

Intercambio Diffie-Hellman de Claves Secretas Usando OpenSSL

Introducción

Contiene un ejemplo del uso de operaciones OpenSSL para realizar un intercambio de clave secreta según el protocolo de Diffie-Hellmen (DHKE).

El objetivo en DHKE es que dos usuarios obtengan una clave secreta compartida, sin que ningún otro usuario conozca esa clave. El intercambio se realiza a través de una red pública, es decir, todos los mensajes enviados entre los dos usuarios pueden ser interceptados y leídos por cualquier otro usuario; no obstante debe ponerse empeño en impedir la actividad típica de un tercero interpuesto, para lo cual podemos apoyarnos en una autoridad de certificación.

El protocolo utiliza aritmética modular y especialmente exponenciales. La seguridad del protocolo se basa en el hecho de que resolver un problema de logaritmo discreto (el inverso de un exponencial) es prácticamente imposible cuando se utilizan valores adecuados lo suficientemente grandes.

DHKE es realizado por dos usuarios, en dos computadoras diferentes. Los pasos realizados por cada usuario son los mismos sólo que con diferentes archivos. En lo que sigue hablaremos del usuario 1 y el usuario 2.

Pasos Para el Intercambio de Claves Con OpenSSL Según el Protocolo Diffie-Hellman

Generación de los parámetros globales públicos de Diffie-Hellman

Con la siguiente orden generaremos los parámetros globales públicos de Diffie-Hellman y los guardaremos en el fichero `dhp.pem`:



```
$ openssl genpkey -genparam -algorithm DH -out dhp.pem
```

lo que ofrece el siguiente diálogo:

```
.....+.....
.....+.....
.....+......+......+.
.....+.
.....
.....+*****
```

Podemos ver el contenido del fichero como habitualmente:

```
$ openssl pkeyparam -in dhp.pem -text
```



lo que mostrará un texto similar al siguiente:

```
-----BEGIN DH PARAMETERS-----
MIGHAoGBAOZVzJ4E8766527Mp3FD71xEUYdmFan4tPcSuP099H7n9xfAm7WytmRQ
```

```

gxNn2dz4X58FKLzVMY+x2rLyP0d8SLa30B7tE+gKFMymswteN//lPbFeLWtyei78
71GJNnjVDpqJFmo1nldMTDy15Z+ueZJP5vGGs2ouvem/Cf5N5QRtAgEC
-----END DH PARAMETERS-----
PKCS#3 DH Parameters: (1024 bit)
prime:
  00:e6:55:cc:9e:04:f3:be:ba:e7:6e:cc:a7:71:43:
  ef:5c:44:51:87:66:15:a9:f8:b4:f7:12:b8:f3:bd:
  f4:7e:e7:f7:17:c0:9b:b5:b2:b6:64:50:83:13:67:
  d9:dc:f8:5f:9f:05:28:bc:d5:31:8f:b1:da:b2:f2:
  3c:e7:7c:48:b6:b7:38:1e:ed:13:e8:0a:14:cc:a6:
  b3:0b:5e:37:ff:e5:3d:b1:5e:2d:6b:72:7a:2e:fc:
  ee:51:89:36:78:d5:0e:9a:89:16:6a:35:9e:57:4c:
  4c:3c:a5:e5:9f:ae:79:92:4f:e6:f1:86:b3:6a:2e:
  bd:e9:bf:09:fe:4d:e5:04:53
generator: 2 (0x2)

```



El fichero **dhp.pem** puede ser compartido en abierto, es decir, sin medidas especiales de seguridad, pero enviándolo al menos firmado según certificados expedidos por una CA fiable.

Generación de las claves públicas y privadas de los dos usuarios

Cada usuario usa ahora los parámetros públicos para generar sus propias claves públicas y privadas, que guardarán en los ficheros **dhkey1.pem** (para el usuario 1) y **dhkey2.pem** (para el usuario 2).

Para ello, el usuario 1 ejecutará la orden:



```
$ openssl genpkey -paramfile dhp.pem -out dhkey1.pem
```

y el usuario 2 su correspondiente:

```
$ openssl genpkey -paramfile dhp.pem -out dhkey2.pem
```

Para ver el contenido de cualquiera de estos dos ficheros podemos ejecutar la orden:

```
$ openssl pkey -in dhkey1.pem -text -noout
```

lo que ofrece el siguiente diálogo:

```

PKCS#3 DH Private-Key: (1024 bit)
private-key:
  48:88:7d:fd:09:0d:17:5e:33:be:ea:29:e7:b3:83:
  34:29:92:89:06:9f:9a:b4:92:b6:78:07:90:5f:aa:
  98:d9:6d:22:d7:92:05:be:f0:3f:14:af:09:3f:17:
  97:b9:04:73:41:32:c3:4a:38:8f:dc:79:e2:04:97:
  bf:a1:46:5f:ec:2a:ac:4f:ab:df:3b:b0:c9:be:86:
  85:d2:0f:7b:fe:03:46:a9:ab:df:7f:a8:98:38:c3:
  fa:9c:a6:ab:db:70:be:a6:67:95:ab:66:99:cc:15:

```



```

4d:b5:94:90:e4:15:9f:14:2f:7b:dd:ff:60:3c:1d:
3d:6c:4f:ff:81:77:e1:1d
public-key:
00:d9:ab:d7:8c:93:df:dd:eb:92:0d:57:d6:51:31:
26:d8:f1:11:8c:92:37:a4:51:01:40:8d:bf:fe:6c:
fd:95:b0:11:a0:16:e4:e0:ab:8a:ef:06:01:e8:36:
a4:52:b8:bb:88:be:7c:a7:1e:4f:22:f9:7a:a6:5f:
83:58:ee:69:34:8d:12:27:d6:5d:b6:e5:36:41:d1:
a6:54:2a:a4:be:4b:4a:dc:75:fa:c8:16:af:79:a8:
e3:f5:09:7f:83:13:e7:b7:25:df:37:ea:dc:8c:77:
4e:20:33:df:a9:9c:95:cc:ef:33:3b:f4:02:b0:66:
19:8c:30:48:1e:2a:83:87:5c
prime:
00:e6:55:cc:9e:04:f3:be:ba:e7:6e:cc:a7:71:43:
ef:5c:44:51:87:66:15:a9:f8:b4:f7:12:b8:f3:bd:
f4:7e:e7:f7:17:c0:9b:b5:b2:b6:64:50:83:13:67:
d9:dc:f8:5f:9f:05:28:bc:d5:31:8f:b1:da:b2:f2:
3c:e7:7c:48:b6:b7:38:1e:ed:13:e8:0a:14:cc:a6:
b3:0b:5e:37:ff:e5:3d:b1:5e:2d:6b:72:7a:2e:fc:
ee:51:89:36:78:d5:0e:9a:89:16:6a:35:9e:57:4c:
4c:3c:a5:e5:9f:ae:79:92:4f:e6:f1:86:b3:6a:2e:
bd:e9:bf:09:fe:4d:e5:04:53
generator: 2 (0x2)

```

Lo anterior es respecto al usuario 1; en cuanto al usuario 2 ejecutaría:

```
$ openssl pkey -in dhkey2.pem -text -noout
```

lo que ofrece el siguiente diálogo:

```

PKCS#3 DH Private-Key: (1024 bit)
private-key:
5d:70:9b:3e:a7:c9:b1:3b:df:17:d3:76:dd:45:f0:
38:6d:be:35:f6:79:5d:05:bf:e2:63:b0:ea:25:00:
61:0a:4c:e2:e4:e7:8e:97:6e:cb:9e:f0:f9:4b:d9:
1c:2e:d6:b1:71:cb:ec:56:a7:2f:b0:af:ff:67:df:
37:e0:d8:8c:ab:5d:ef:3d:27:c5:5a:a6:8d:49:30:
6b:4e:d4:1f:5c:40:da:35:d0:bc:c7:3d:16:a3:13:
2e:86:af:13:8b:65:c4:19:f2:75:43:e7:11:b6:5a:
81:d1:e0:ff:5d:f3:c2:f4:6f:d2:f0:72:97:66:b9:
93:3d:17:b0:06:ef:8a:3b
public-key:
00:d9:9a:00:1b:98:f5:0b:e2:d6:57:f7:4d:e3:4b:
aa:43:ad:e2:f2:93:31:a1:e7:4b:a7:06:dc:ab:22:
09:5a:0d:41:1a:c1:37:c0:6d:88:f4:7c:0a:22:27:
1e:d3:84:39:51:92:62:d5:14:9e:68:ee:2f:69:27:
ae:dd:d1:e6:a2:5f:3c:d2:7b:a7:7c:8e:61:28:fb:
8b:1c:d7:a0:0b:d3:7b:37:af:78:b2:7e:eb:62:a7:

```

```

85:b6:0f:90:10:b7:9c:ce:ec:84:a9:28:e3:7f:22:
8f:76:cd:68:58:56:45:fd:3e:36:37:a1:99:aa:ca:
4a:65:65:af:a8:21:ee:1f:b6
prime:
00:e6:55:cc:9e:04:f3:be:ba:e7:6e:cc:a7:71:43:
ef:5c:44:51:87:66:15:a9:f8:b4:f7:12:b8:f3:bd:
f4:7e:e7:f7:17:c0:9b:b5:b2:b6:64:50:83:13:67:
d9:dc:f8:5f:9f:05:28:bc:d5:31:8f:b1:da:b2:f2:
3c:e7:7c:48:b6:b7:38:1e:ed:13:e8:0a:14:cc:a6:
b3:0b:5e:37:ff:e5:3d:b1:5e:2d:6b:72:7a:2e:fc:
ee:51:89:36:78:d5:0e:9a:89:16:6a:35:9e:57:4c:
4c:3c:a5:e5:9f:ae:79:92:4f:e6:f1:86:b3:6a:2e:
bd:e9:bf:09:fe:4d:e5:04:53
generator: 2 (0x2)

```

Los ficheros `dhkey1.pem` y `dhkey2.pem` deben ser guardados celosamente.

Extracción de claves públicas.

Cada usuario debe extraer ahora su clave pública. El usuario 1 ejecutará a tal fin:



```
$ openssl pkey -in dhkey1.pem -pubout -out dhp1.pem
```

y podrá ver el contenido de lo generado con la orden:

```
$ openssl pkey -pubin -in dhp1.pem -text
```

que le ofrece lo siguiente:

```

-----BEGIN PUBLIC KEY-----
MIIBIDCB1QYJKoZIhvcNAQMBMIGHAoGBA0ZVzJ4E8766527Mp3FD71xEUYdmFan4
tPcSuP099H7n9xfAm7WytmRQgxNn2dz4X58FKLzVMY+x2rLyP0d8SLa30B7tE+gK
FMymswteN//lPbFeLWtyei7871GJNnjVDpqJFmo1nldMTDyl5Z+ueZJP5vGGs2ou
vem/Cf5N5QRTAgECA4GFAAKBgQDZq9eMk9/d65INV9ZRMsbY8RGMkjekUQFAjb/+
bP2VsBGgFuTgq4rvBgHoNqRSuLuIvnynHk8i+XqmX4NY7mk0jRIIn1l225TZBoaZU
KqS+S0rcdfrIFq95qOP1CX+DE+e3Jd836tyMd04gM9+pnJXM7zM79AKwZhmMMEge
KoOHXA==
-----END PUBLIC KEY-----
PKCS#3 DH Public-Key: (1024 bit)
public-key:
00:d9:ab:d7:8c:93:df:dd:eb:92:0d:57:d6:51:31:
26:d8:f1:11:8c:92:37:a4:51:01:40:8d:bf:fe:6c:
fd:95:b0:11:a0:16:e4:e0:ab:8a:ef:06:01:e8:36:
a4:52:b8:bb:88:be:7c:a7:1e:4f:22:f9:7a:a6:5f:
83:58:ee:69:34:8d:12:27:d6:5d:b6:e5:36:41:d1:
a6:54:2a:a4:be:4b:4a:dc:75:fa:c8:16:af:79:a8:
e3:f5:09:7f:83:13:e7:b7:25:df:37:ea:dc:8c:77:

```

```

4e:20:33:df:a9:9c:95:cc:ef:33:3b:f4:02:b0:66:
19:8c:30:48:1e:2a:83:87:5c
prime:
00:e6:55:cc:9e:04:f3:be:ba:e7:6e:cc:a7:71:43:
ef:5c:44:51:87:66:15:a9:f8:b4:f7:12:b8:f3:bd:
f4:7e:e7:f7:17:c0:9b:b5:b2:b6:64:50:83:13:67:
d9:dc:f8:5f:9f:05:28:bc:d5:31:8f:b1:da:b2:f2:
3c:e7:7c:48:b6:b7:38:1e:ed:13:e8:0a:14:cc:a6:
b3:0b:5e:37:ff:e5:3d:b1:5e:2d:6b:72:7a:2e:fc:
ee:51:89:36:78:d5:0e:9a:89:16:6a:35:9e:57:4c:
4c:3c:a5:e5:9f:ae:79:92:4f:e6:f1:86:b3:6a:2e:
bd:e9:bf:09:fe:4d:e5:04:53
generator: 2 (0x2)

```

El usuario 2 ejecutará la orden:

```
$ openssl pkey -in dhkey2.pem -pubout -out dhp2.pem
```

y podrá ver el contenido de lo generado con la orden:

```
$ openssl pkey -pubin -in dhp2.pem -text
```

Seguidamente los usuarios intercambiarán entre ellos los ficheros `dhp1.pem` y `dhp2.pem` eventualmente firmados mediante certificado emitido por una CA fiable. Hecho esto, cada usuario podrá obtener su secreto compartido.

Obtención del secreto compartido

El usuario 1 ejecutará lo siguiente:

```
$ openssl pkeyutl -derive -inkey dhkey1.pem -peerkey dhp2.pem -out secret1.bin
```

para obtener el secreto compartido, un valor binario de 128 bits albergado en el fichero `secret1.bin`. El usuario 2 ejecutará la orden:

```
$ openssl pkeyutl -derive -inkey dhkey2.pem -peerkey dhp1.pem -out secret2.bin
```

utilizando su clave privada y la clave pública del usuario 1, obteniendo `secret2.bin`. Los ficheros `secret1.bin` y `secret2.bin` deben ser iguales, lo que podemos comprobar con la orden:

```
$ diff secret1.bin secret2.bin
```

o también

```
$ cmp secret1.bin secret2.bin
```

y si queremos constatar visulamente la coincidencia, podemos hacer:

```

$ xxd secret1.bin
0000000: b7cb b892 b541 7810 d8ec d089 6c89 3c19  ....Ax.....l.<.
0000010: e8e1 27d8 66ee dac8 684a f0bd 0a7f e7d3  ..'.f...hJ.....
0000020: 3643 8654 fddf 4399 e58e 2c7c 3d33 9532  6C.T..C...,|=3.2

```

```

0000030: f693 edf2 c9a0 40e8 58b8 38de 74a5 c0b0 .....@.X.8.t...
0000040: 64ab 4006 a3cd d795 2cef d0fc 2b0f d1ab d.@.....,....+...
0000050: d1e5 1a2a 3431 e3fa ba63 f7cf 1c61 ff65 ...*41...c...a.e
0000060: d9cd c85d c5fe 5c50 c543 aaeb de49 8501 ...]...\P.C...I..
0000070: 6cf1 66a6 87b6 ddec 835c b4b1 3d9d e2fe l.f.....\..=...
$ xxd secret2.bin
0000000: b7cb b892 b541 7810 d8ec d089 6c89 3c19 .....Ax.....l.<.
0000010: e8e1 27d8 66ee dac8 684a f0bd 0a7f e7d3 ..'.f...hJ.....
0000020: 3643 8654 fddf 4399 e58e 2c7c 3d33 9532 6C.T..C...,|=3.2
0000030: f693 edf2 c9a0 40e8 58b8 38de 74a5 c0b0 .....@.X.8.t...
0000040: 64ab 4006 a3cd d795 2cef d0fc 2b0f d1ab d.@.....,....+...
0000050: d1e5 1a2a 3431 e3fa ba63 f7cf 1c61 ff65 ...*41...c...a.e
0000060: d9cd c85d c5fe 5c50 c543 aaeb de49 8501 ...]...\P.C...I..
0000070: 6cf1 66a6 87b6 ddec 835c b4b1 3d9d e2fe l.f.....\..=...

```

Ejemplo de Uso Tras Haber Compartido la Clave

El verdadero uso de la clave que resulta de compartir un fichero es tunelar un flujo de información al estilo de como opera SSH.

Aquí en esta práctica nos limitaremos a ilustrar la operación con un supuesto menos realista y, en realidad, poco útil. El objetivo es simplemente practicar con el cifrado simétrico según las posibilidades que ofrece OpenSSL.

La idea es contar con un fichero `confidential.txt` que ha de permanecer cifrado y que sólo pueden alterar los usuarios que compartieron la clave. Supondremos una estructura de directorios como la que sugiere lo siguiente:

```

$ ls
confidential_file  user_01  user_02

```

Creamos el fichero:



```
$ echo Ésta es mi aportación de hoy. > confidential_file/confidential.txt
```

que cifrará el usuario 1 con la clave compartida. Esto puede llevarlo a cabo de la siguiente forma:

```

$ openssl enc -e -aes256 -base64 -md sha256 \
    -pass file:./user_01/secret1.bin \
    -pbkdf2 -iter 100000 -salt \
    -in confidential_file/confidential.txt \
    -out confidential_file/confidential.txt.enc

```

y por algún medio borraremos sin dejar rastro el fichero `confidential.txt`, por ejemplo (cuidado con esta orden en ordenadores con memoria de estado sólido, quédese sólo con la idea):

```
$ srm confidential_file/confidential.txt
```

El usuario 2 podría hacer ahora su aportación descifrando:

```
$ openssl enc -d -aes256 -base64 -md sha256 \  
-pass file:./user_02/secret2.bin \  
-pbkdf2 -iter 100000 -salt \  
-in confidential_file/confidential.txt.enc \  
-out confidential_file/confidential.txt
```

escribiendo en el fichero:

```
$ echo La de mañana, es devenir. >> confidential_file/confidential.txt
```

y volviendo a cifrar:

```
$ openssl enc -e -aes256 -base64 -md sha256 \  
-pass file:./user_02/secret2.bin \  
-pbkdf2 -iter 100000 -salt \  
-in confidential_file/confidential.txt \  
-out confidential_file/confidential.txt.enc
```

Una forma alternativa de cifrar y descifrar con la clave compartida es mediante la orden:

```
$ openssl enc -e -aes256 -base64 -md sha256 \  
-kfile ./user_01/secret1.bin \  
-pbkdf2 -iter 100000 -salt \  
-in confidential_file/confidential.txt \  
-out confidential_file/confidential.txt.enc
```

para poder cifrar el usuario 1 con la clave compartida y mediante la orden:

```
$ openssl enc -d -aes256 -base64 -md sha256 \  
-kfile ./user_02/secret2.bin \  
-pbkdf2 -iter 100000 -salt \  
-in confidential_file/confidential.txt.enc \  
-out confidential_file/confidential.txt
```

para poder descifrar el usuario 2 con la clave compartida.